Step 2.1: read one file (a test run on one station)

Type	File	What it is
Code	src/data/read_cpcb.py	read_one(): reads one CPCB Excel file and its station details
Output	temp/2.1_one_file.csv	the result for one file: Ballygunge 2025, 7,388 rows × 21 columns
Input	Kolkata/<station>/*.xlsx	the raw files (never changed)

Step 2.2: read all files onto one hourly grid

Type	File	What it is
Code	src/data/read_all.py	read_all(): reads all 21 files and puts every station on the same hourly timeline
Output	data/processed/cpcb_hourly.csv	the main table: 184,128 rows (7 stations × 26,304 hours)

Step 2.3: how many hours have a value

Type	File	What it is
Code	src/data/availability.py	counts hours with a value
Chart	figures/availability_targets.png	PM2.5 and PM10 hours per station and year
Table	figures/availability.csv	% of hours present per station, year and variable

Step 2.4: stuck sensors (same PM value for 12 h or more)

Type	File	What it is
Code	src/clean/flags.py	finds stuck runs (count only)
Chart	figures/flag_check.png	stuck hours per station and year
Table	figures/flat_runs.csv	every stuck run: station, start, end, length, value
Table	figures/flag_counts.csv	stuck hours per station, year and variable

Step 2.5: CPCB range check + stuck marks + drop columns

Type	File	What it is
Code	src/clean/range_clean.py	counts out-of-range values, adds the 0/1 stuck marks, drops columns
Chart	figures/range_check.png	in-range vs out-of-range values per station, year and variable
Table	figures/range_counts.csv	the numbers behind the chart
Output	data/processed/cpcb_clean.csv	184,128 rows; values unchanged; plus stuck_pm25, stuck_pm10; minus XYLENE, BENZENE, TOLUENE, RF, CO

Step 2.6: ERA5 download (copied, not run)

Type	File	What it is
Input	data/raw/era5/era5_2023_01.nc … era5_2025_12.nc	**36 monthly files**, copied from PROJECT. No new download.

Step 2.7: attach wind to each station

Type	File	What it is
Code	src/weather/merge_era5.py	picks u10, v10 → computes wind speed → joins to each station
Input	data/stations.csv	latitude and longitude of the 7 stations
Input	data/processed/cpcb_clean.csv	from step 2.5
Output	data/processed/merged.csv	**the final table before Part 3**: 184,128 rows × 21 columns

Step 3.1: year split plan (GATE)

Type	File	What it is
Code	src/splits/plan.py	counts hours and days per split and season, checks the size rules
Code	src/splits/weeks.py	odd / even week split of 2024, used by every later check

Step 3.2: features

Type	File	What it is
Code	src/features/build.py	6-hour fill, lags, 24-hour means, sin/cos, seasons
Test	tests/test_features_no_leak.py	proves no feature looks at the future
Output	data/processed/features.csv	184,128 × 41 (35 features + targets + IDs)

Step 3.3: Model inputs

Type	File	What it is
Code	**src/features/samples.py**	**log → scale → 48-hour windows → miss flags**
Test	**tests/test_samples.py**	**answers come after inputs, no window crosses years, scaler uses 2023 only**
Inputs	**artifacts/samples/X_<train/calib/test>.npy**	**windows of 48 h × 38 columns**
Answers	**y_*.npy (scaled), y_raw_*.npy (µg/m^3)**	**2 targets × 4 horizons per window**
Readable	**artifacts/samples/meta_*.csv**	**station, time, season and the 8 answers in µg/m^3**
Model	**scaler.joblib, feature_names.json, target_layout.json**	**the 2023 scaler, the 38 column names, the answer order**
View	**data/processed/model_table.csv**	**the 38 inputs per hour, exactly as the model sees them**

Part 4: the 5 forecasters

Code

File	Job
src/models/forecasters.py	builds the 5 models (GRU, LSTM, 1D CNN, CNN+LSTM, simplified TFT) and the 6-number embedding
src/models/train.py	trains one model at a time and saves its outputs (--model gru --kind point)
src/models/metrics.py	RMSE, MSE, MAE
src/models/compare.py	makes all the comparison tables and charts
src/models/summary.py	writes the model.summary() layer tables
tests/test_forecasters.py	checks outputs read only the embedding, and low ≤ median ≤ high

Saved models: artifacts/forecasters/<model>/ (one folder per model)

Folder	Files	What they hold
point/	model.keras	the trained single-number model
	history.csv	loss per epoch (training and validation)
	calib_pred.csv	its 2024 forecasts (the Part 4 scoreboard)
	report.txt	the printed check
quantile/	model.keras, history.csv, report.txt, tag.txt	the low/median/high model, its loss and its version tag
	train.csv, calibration.csv, test.csv	every forecast + range + embedding for 2023, 2024, 2025: the files the rest of the pipeline uses

Charts and tables, all models together: figures/

File	Walkthrough step
forecast_errors.png + forecast_errors.csv	Step 2: errors vs "no change"
forecast_skill.png	Step 2: improvement %
forecast_loss.png	Step 3: training curves
forecast_coverage.png + forecast_quantile.csv	Step 4: did 90% land inside the band
forecast_band_pm25.png, forecast_band_pm10.png	Step 4: picture of the band
forecast_summaries.txt	the layer tables (model.summary())

Charts per model: figures/<model>/

File	Walkthrough step
forecast_errors.png, forecast_skill.png, forecast_loss.png, forecast_coverage.png, forecast_band_pm25/pm10.png	Steps 2–4 for this model only
forecast_errors_station_year_pm25/pm10.png + errors_station_year.csv	Step 5: errors per station and ye

Part 5: Worker B, "is this hour unusual?"

Code: src/worker_b/

File	Step	Job
distance.py	5.2	season shapes (2023) + distance of each hour
pvalue.py	5.3	distance → p-value → s_embed; the odd/even week check
smooth.py	5.5	smoothing over time + toy test
raw_score.py	5.6	raw-input distance → s_raw; fused = max
labels.py	5.6b	final score s → Green / Yellow / Red
plots.py	all	every Part 5 chart

Saved data: artifacts/forecasters/<model>/worker_b/ (per model)

File	Step	Holds
embed_shapes.npz	5.2	centre + shape of each season's cloud (fitted on 2023)
distances.csv	5.2	distance of every 2023 and 2024 hour
pvalue_check.csv	5.3	share with p < 0.10 / 0.05 / 0.02 (the "12 of 100" numbers)
raw_check.csv	5.6	even-week hours: s_raw, s_embed, fused
label_check.csv, label_check_rows.csv	5.6b	colour shares on check weeks + every row behind them
worker_b_calibration.csv	5.6b	every 2024 hour: scores, final s, colour per horizon
worker_b_test.csv	7.1	every 2025 hour: the same columns

Shared by all models: artifacts/forecasters/raw_shapes.npz (5.6, the raw-input season shapes).

Part 6: Worker C (the ranges)

File	What it is
src/worker_c/methods.py	the 4 ways to make a range: split, CQR, adaptive, adaptive by season

7.1: Final decision on 2025

File	What it is
src/decision/decide.py	the code
artifacts/forecasters/<model>/worker_b/worker_b_test.csv	Worker B scores and labels for each 2025 hour (same columns as the 2024 file)
artifacts/forecasters/<model>/decisions_test.csv	the flag (Green/Yellow/Red), forecast, and base and Yellow ranges for each hour

7.1: Final decision on 2025

File	What it is
src/decision/decide.py	the code
artifacts/forecasters/<model>/worker_b/worker_b_test.csv	Worker B scores and labels for each 2025 hour (same columns as the 2024 file)
artifacts/forecasters/<model>/decisions_test.csv	the flag (Green/Yellow/Red), forecast, and base and Yellow ranges for each hour

7.2: Coverage on normal hours and smoggy hours

File	What it is
src/eval/stratified.py	the code
figures/<model>/stratified_table1.csv	coverage on normal hours vs the top 5% smoggy hours
figures/<model>/stratified_table2.csv	coverage per season
figures/<model>/stratified_table3.csv	coverage per station
figures/<model>/stratified_coverage_{pm25,pm10}.png	bars for the 6 methods, normal vs smoggy
figures/<model>/stratified_station_{pm25,pm10}.png	coverage per station
figures/stratified_compare_{pm25,pm10}.png	all 5 models side by side